

Modeling and Verification of a
Data Aligner
using **SystemVerilog** and **UVM**

Verification Plan

by

Hossam Shaher Anis

Cairo, 2026

Contents

Verification Plan..... 1

Contents 2

List of Tables..... 3

1 Introduction..... 4

2 Test Plan 5

3 Coverage Plan..... 6

4 Assertion Plan..... 7

List of Tables

Table 1 Test plan..... 5

Table 2 Coverage plan..... 6

Table 3 Assertion plan..... 7

1 Introduction

The **Data Aligner** module takes in an unaligned stream of data and outputs it as an aligned stream of data based on its configuration.¹

In [this project](#), the Data Aligner is modeled and verified using the SystemVerilog HDVL and the Universal Verification Methodology (UVM).

This project is the course project of this Udemmy course: [Design Verification with SystemVerilog/UVM](#).

This document contains three plans:

1. Test plan.
2. Coverage plan.
3. Assertion (checking) plan.

¹ For more information about the Data Aligner module (its interfaces, registers, functionality, and more), see its specification: *Aligner Datasheet - v1.1*.

2 Test Plan

Table 1 shows the test plan.

Table 1 Test plan

# ²	Test Case Name ³	Test Feature ⁴	Sequence ⁵
1	algn_test_reg_access	Accessing registers: Read followed by write (loop)	Insert all mapped registers in a SystemVerilog queue. Repeat the following sequence of actions num_access times: Shuffle the queue, read the 0 th register, write a random value in it, then wait for delay.
2	algn_test_random	Send random transactions on the Rx interface and the Tx interface	Drive random transactions on the Tx interface continuously in the background (using a fork - join_none). Write randomize values in configuration registers. Repeat the following sequence of actions num_md_rx_transactions times: Drive random transactions on the Rx interface, wait, then read the status register.
3	algn_test_illegal_rx	Send <i>illegal</i> random transactions on the Rx interface	Identical to the previous sequence, but with one exception: Random transactions to be driven on the Rx interface shall be <i>illegal</i> .

² # = Serial number.

³ Test case name = Test *scenario* name = UVM test name.

⁴ Test feature = Objective of the scenario = Simple description of the scenario.

⁵ Sequence = Detailed steps for stimulus generation for the scenario.

3 Coverage Plan

[Table 2](#) shows the coverage plan.

Table 2 Coverage plan

#	File Name ⁶	covergroup	Coverpoint/ cross	Description
1	apb_coverage_collector	cover_reg	CTRL_OFFSET	Value of CTRL.OFFSET
			CTRL_SIZE	Value of CTRL.SIZE
			CTRL_OFFSET_x_CTRL_SIZE	Cross of the previous two cover points
			IRQ_MAX_DROP	Value of IRQ.MAX_DROP
2	md_coverage_collector	cover_item	offset	Offset of the MD access
			size	Size of the MD access
			offset_x_size	Cross of the previous two cover points
			response	Response of the MD access

⁶ File Name = Coverage collector name

4 Assertion Plan

[Table 3](#) shows the assertion plan.

Table 3 Assertion plan

#	Assertion ID	Description	File Name ⁷
1	PENABLE_ENTERING_SETUP_PHASE_A	PENABLE must be asserted in the second cycle of the transfer	apb_if
2	PENABLE_EXITING_SETUP_PHASE_A	PENABLE must be deasserted at the end of the transfer	
3	PENABLE_STABLE_AT_ACCESS_PHASE_A	Master-driven must remain constant throughout the transfer	
4	PWRITE_STABLE_AT_ACCESS_PHASE_A		
5	PADDR_STABLE_AT_ACCESS_PHASE_A		
6	PWDATA_STABLE_AT_ACCESS_PHASE_A		
7	UNKNOWN_VALUE_PSEL_A	APB signals cannot have unknown values	
8	UNKNOWN_VALUE_PENABLE_A		
9	UNKNOWN_VALUE_PWRITE_A		
10	UNKNOWN_VALUE_PADDR_A		
11	UNKNOWN_VALUE_PWDATA_A		
12	UNKNOWN_VALUE_PRDATA_A		
13	UNKNOWN_VALUE_PREADY_A		
14	UNKNOWN_VALUE_PSLVERR_A		
15	INFINITE_LENGTH_APB_A	APB transfer cannot have an infinite length	apb_monitor

⁷ File Name = Interface/ monitor name

Table 3 Assertion plan (*cont.*)

#	Assertion ID	Description	File Name
16	DATA_WIDTH_POWER_OF_2_A	DATA_WIDTH must be a power of 2.	md_if
17	DATA_WIDTH_BIGGER_THAN_OR_EQUAL_8_A	Minimum legal value of DATA_WIDTH is 8.	
18	VALID_HIGH_UNTIL_READY_A	Once VALID becomes high, it must stay high until READY becomes high	
19	UNKNOWN_VALUE_DATA_A	DATA is valid while VALID is high	
20	STABLE_DATA_UNTIL_READY_A	DATA must remain constant until READY becomes high	
21	UNKNOWN_VALUE_OFFSET_A	OFFSET is valid while VALID is high	
22	STABLE_OFFSET_UNTIL_READY_A	OFFSET must remain constant until READY becomes high	
23	UNKNOWN_VALUE_SIZE_A	SIZE is valid while VALID is high	
24	STABLE_SIZE_UNTIL_READY_A	SIZE must remain constant until READY becomes high	
25	SIZE_EQ_0_A	SIZE cannot have value 0	
26	UNKNOWN_VALUE_ERR_A	ERR is valid only when both VALID and READY are high	
27	ERR_HIGH_AT_VALID_AND_READY_A	ERR can be high only when VALID and READY are high	
28	UNKNOWN_VALUE_VALID_A	VALID cannot have an unknown value	
29	UNKNOWN_VALUE_READY_A	READY is valid while VALID is high	
30	SIZE_PLUS_OFFSET_GT_DATA_WIDTH_A	OFFSET + SIZE cannot be bigger than the data width in bytes	
31	INFINITE_LENGTH_MD_A	MD transfer cannot have an infinite length	md_monitor

Notes:

- There are two instances of md_if: (1) md_rx_if, and (2) md_tx_if.
- There are two instances of md_monitor: one in md_agent_master and one in md_agent_slave.